Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

LAB NO 9

DATA STRUCTURES AND ALGORITHMS

OBJECTIVE: To implement Stack as Arrays and linked list and Implement Stack by using Stack class.

LAB TASKS :

Task 1:

```
import java.util.Stack;
import java.util.Scanner;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Initialize stack and input
        Stack<String> stack = new Stack<>();

        // Example of adding colors to the stack
        System.out.println("Enter the number of balls to add to the stack:");
        int n = sc.nextInt();
        sc.nextLine(); // To consume the newline character

        System.out.println("Enter the colors of balls (red, yellow, orange, green):");

        // Read n colors and add them to the stack
        for (int i = 0; i < n; i++) {
            String color = sc.nextLine().trim().toLowerCase();
            stack.push(color);
        }

        // Display the original stack
        System.out.println("Original stack:");
        printStack(stack);

        // Get the color that the user likes
        System.out.println("Enter the color of the ball you want to remove:");
        String likedColor = sc.nextLine().trim().toLowerCase();

        // Create a temporary stack to hold balls while removing the liked color
        Stack<String> tempStack = new Stack<>();

        // Remove the liked color balls and store others in tempStack
        while (!stack.isEmpty()) {
            String color = stack.pop();
            if (!color.equals(likedColor)) {
```

```
                tempStack.push(color);
            }
        }

        // Rebuild the stack in the original order minus the liked color
        while (!tempStack.isEmpty()) {
            stack.push(tempStack.pop());
        }

        // Display the updated stack
        System.out.println("Updated stack:");
        printStack(stack);

        sc.close();
    }

    // Method to print stack elements
    public static void printStack(Stack<String> stack) {
        if (stack.isEmpty()) {
            System.out.println("The stack is empty.");
        } else {
            for (String color : stack) {
                System.out.print(color + "->");
            }
            System.out.println();  // Newline after stack
        }
    }
```

Output:

```
Enter the number of balls to add to the stack:
5
Enter the colors of balls (red, yellow, orange, green):
yellow
orange
green
red

Original stack:
yellow->orange->green->red->->
Enter the color of the ball you want to remove:
orange
Updated stack:
yellow->green->red->->

=== Code Execution Successful ===
```

Task 2:

Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

```java
import java.util.Scanner;

// Define a Node class to represent each element in the linked list
class Node {
    int data;
    Node next;

    // Constructor
    public Node(int data) {
        this.data = data;
        this.next = null;
    }
}

// Stack class to represent the stack
class Stack {
    private Node top; // The top of the stack

    // Constructor to initialize an empty stack
    public Stack() {
        top = null;
    }

    // Push operation (insert element onto stack)
    public void push(int value) {
        Node newNode = new Node(value);
        newNode.next = top;
        top = newNode;
        System.out.println("Pushed " + value + " onto stack.");
    }

    // Pop operation (remove element from stack)
    public int pop() {
        if (isEmpty()) {
            System.out.println("Stack is empty. Cannot pop.");
            return -1;
        }
```

Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

```
        int poppedValue = top.data;
        top = top.next; // Move the top pointer to the next node
        return poppedValue;
    }

    // Peek operation (view the top element without removing)
    public int peek() {
        if (isEmpty()) {
            System.out.println("Stack is empty.");
            return -1;
        }
        return top.data;
    }

    // Check if the stack is empty
    public boolean isEmpty() {
        return top == null;
    }

    // Display the elements in the stack
    public void display() {
        if (isEmpty()) {
            System.out.println("Stack is empty.");
            return;
        }
        Node current = top;
        System.out.print("Stack: ");
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.next;
        }
        System.out.println();
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        int choice = sc.nextInt();

        switch (choice) {
            case 1:
                System.out.print("Enter value to push: ");
                int value = sc.nextInt();
                stack.push(value);
                break;

            case 2:
                int poppedValue = stack.pop();
                if (poppedValue != -1) {
                    System.out.println("Popped value: " + poppedValue);
                }
                break;

            case 3:
                int topValue = stack.peek();
                if (topValue != -1) {
                    System.out.println("Top value is: " + topValue);
                }
                break;

            case 4:
                stack.display();
                break;

            case 5:
                System.out.println("Exiting...");
                sc.close();
                return;

            default:
                System.out.println("Invalid choice. Please try again.");
        }
    }
```

Output:

```
Choose an operation:
1. Push
2. Pop
3. Peek
4. Display Stack
5. Exit
Enter your choice: 1
Enter value to push: 20
Pushed 20 onto stack.

Choose an operation:
1. Push
2. Pop
3. Peek
4. Display Stack
5. Exit
Enter your choice: 2
Popped value: 20
```

Task 3:

```
import java.util.Scanner;
import java.util.Stack;

public class Main {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Create a stack of 10 elements
        Stack<Integer> stack = new Stack<>();

        // Push 10 elements onto the stack
        System.out.println("Pushing 10 elements onto the stack...");
        for (int i = 1; i <= 10; i++) {
            stack.push(i);
        }

        // Display the stack after pushing 10 elements
        System.out.println("Stack after pushing 10 elements:");
        displayStack(stack);

        // Perform five additional operations
        while (true) {
            System.out.println("\nChoose an operation:");
            System.out.println("1. Push a new element");
            System.out.println("2. Pop the top element");
            System.out.println("3. Peek the top element");
            System.out.println("4. Display the stack");
            System.out.println("5. Exit");
            System.out.print("Enter your choice: ");
            int choice = sc.nextInt();

            switch (choice) {
                case 1:
                    // Push a new element
                    System.out.print("Enter a value to push onto the stack: ");
                    int value = sc.nextInt();
```

Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

```
            stack.push(value);
            System.out.println("Pushed " + value + " onto the stack.");
            break;

        case 2:
            // Pop the top element
            if (!stack.isEmpty()) {
                int poppedValue = stack.pop();
                System.out.println("Popped value: " + poppedValue);
            } else {
                System.out.println("Stack is empty. Cannot pop.");
            }
            break;

        case 3:
            // Peek the top element
            if (!stack.isEmpty()) {
                int topValue = stack.peek();
                System.out.println("Top value is: " + topValue);
            } else {
                System.out.println("Stack is empty.");
            }
            break;

        case 4:
            // Display the stack
            System.out.println("Current stack:");
            displayStack(stack);
            break;

        case 5:
            // Exit the program
            System.out.println("Exiting...");
            sc.close();
            return;

        default:
```

```
            }
        }
    }

    // Method to display the elements in the stack
    public static void displayStack(Stack<Integer> stack) {
        if (stack.isEmpty()) {
            System.out.println("The stack is empty.");
        } else {
            System.out.print("Stack: ");
            for (int i : stack) {
                System.out.print(i + " ");
            }
            System.out.println();
        }
    }
```

Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

Output:

```
Pushing 10 elements onto the stack...
Stack after pushing 10 elements:
Stack: 1 2 3 4 5 6 7 8 9 10

Choose an operation:
1. Push a new element
2. Pop the top element
3. Peek the top element
4. Display the stack
5. Exit
Enter your choice: 1
Enter a value to push onto the stack: 12
Pushed 12 onto the stack.

Choose an operation:
1. Push a new element
2. Pop the top element
3. Peek the top element
4. Display the stack
5. Exit
Enter your choice: 3
Top value is: 12
```

Home Task:

```java
import java.util.Scanner;

// Node class for Linked List implementation of stack
class Node {
    char data;
    Node next;

    public Node(char data) {
        this.data = data;
        this.next = null;
    }
}

// Stack class to handle the operations of a stack using a linked list
class Stack {
    private Node top;

    public Stack() {
        top = null;
    }

    // Push operation to add element to the stack
    public void push(char data) {
        Node newNode = new Node(data);
        newNode.next = top;
        top = newNode;
    }

    // Pop operation to remove and return the top element from the stack
    public char pop() {
        if (isEmpty()) {
            return '$'; // Return a special character to indicate stack underflow
        }
        char item = top.data;
        top = top.next;
        return item;
    }
```

```
    public char peek() {
        if (isEmpty()) {
            return '$';
        }
        return top.data;
    }

    // Check if the stack is empty
    public boolean isEmpty() {
        return top == null;
    }
}

public class Main {

    // Method to return precedence of operators
    public static int precedence(char c) {
        switch (c) {
            case '+':
            case '-':
                return 1;
            case '*':
            case '/':
                return 2;
            case '|':
                return 3;
            default:
                return -1;
        }
    }

    // Method to convert infix to postfix
    public static String infixToPostfix(String infix) {
        Stack stack = new Stack();
        StringBuilder postfix = new StringBuilder();

        for (int i = 0; i < infix.length(); i++) {
```

```
            is encountered
        else if (current == ')') {
            while (!stack.isEmpty() && stack.peek() != '(') {
                postfix.append(stack.pop());
            }
            stack.pop(); // Remove the '(' from the stack
        }
        // If the current character is an operator
        else {
            while (!stack.isEmpty() && precedence(current) <= precedence(stack.peek())) {
                postfix.append(stack.pop());
            }
            stack.push(current);
        }
    }

    // Pop all remaining operators from the stack
    while (!stack.isEmpty()) {
        postfix.append(stack.pop());
    }

    return postfix.toString();
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Input infix expression
    System.out.println("Enter infix expression:");
    String infix = sc.nextLine().trim();

    // Convert to postfix and display
    String postfix = infixToPostfix(infix);
    System.out.println("Postfix expression: " + postfix);

    sc.close();
}
```

Name: WAQAR RIASAT ALI Section: E Roll num: 2023F - BSE - 221

Output:

```
Enter infix expression:
A + ( B * C - ( D / E | F ) * G ) * H
Postfix expression: A+ ( B * C - ( D / E | F  * G  * H

=== Code Execution Successful ===
```

Home Task

```java
import java.util.Scanner;
public class Main {

    // Method to evaluate the postfix expression using stack
    public static int evaluatePostfix(String postfix) {
        // Stack to hold the operands
        int[] stack = new int[postfix.length()];
        int top = -1; // Stack pointer

        // Iterate through each character in the postfix expression
        for (int i = 0; i < postfix.length(); i++) {
            char ch = postfix.charAt(i);

            // If the character is a digit (operand)
            if (Character.isDigit(ch)) {
                // Push the operand onto the stack
                stack[++top] = ch - '0'; // Convert char to int
            }
            // If the character is an operator
            else if (ch == ' ' || ch == '\t') {
                continue; // Skip spaces/tabs between operands and operators
            }
            else {
                // Pop two operands from the stack
                int operand2 = stack[top--];
                int operand1 = stack[top--];

                // Apply the operator
                int result = 0;
                switch (ch) {
                    case '+':
                        result = operand1 + operand2;
                        break;
                    case '-':
                        result = operand1 - operand2;
                        break;
                    case '*':
```

```
                    result = operand1 - operand2;
                    break;
                case '*':
                    result = operand1 * operand2;
                    break;
                case '/':
                    result = operand1 / operand2;
                    break;
                default:
                    System.out.println("Invalid operator");
                    return -1;
            }

            // Push the result back onto the stack
            stack[++top] = result;
        }
    }

    // The result is the only element left in the stack
    return stack[top];
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    // Input the postfix expression
    System.out.println("Enter a postfix expression (e.g., 2 3 1 * + 9 -): ");
    String postfix = sc.nextLine().trim();

    // Evaluate the postfix expression
    int result = evaluatePostfix(postfix);

    // Output the result
    System.out.println("The result of the postfix expression is: " + result);

    sc.close();
}
```

Output:

```
Enter a postfix expression (e.g., 2 3 1 * + 9 -):
2   3 1 * + 9 -
Invalid operator
The result of the postfix expression is: -1

=== Code Execution Successful ===
```